

Integrated Library Management System (CDM ILMS)

Master Capstone Defense Study Manuscript & System Feature Alignment Guide

Document Type: Defense Study Manuscript	System Version: v2.4 (Production Ready)
Target Audience: Project Defense Team	Coverage: 100% Feature & Code Alignment
Primary Stack: React + TypeScript + Node.js + SQLite	Institutes: ICS, ITE, IBE (All Programs)

1. Executive Summary & System Alignment

The **Colegio de Montalban Integrated Library Management System (CDM ILMS)** is a centralized, enterprise-grade dual-platform system designed to modernize library operations, inventory cataloging, student borrowing, and security workflows across all academic institutes (ICS, ITE, IBE). The system bridges automated librarian administrative controls with a student mobile self-service Progressive Web App (PWA).

Manuscript Objective / Requirement	Actual Implemented System Feature	Technical Verification File
Standardized Academic Cataloging	MARC 21 Tagging Standards (001, 020, 082, 100, 245) + LC Call Numbers across ICS, ITE, IBE	src/app/App.tsx server/db.js
Real-Time Catalog Search & Filter	Instant multi-criteria search, ⌘ keyboard focus shortcut, institute segmented control pill group, active filter chips	CatalogPage useMemo hook
Borrow & Inventory Control	Real-time copy availability validation (available/total), borrow ledger logging, active overdue calculation	server/server.js /api/borrow
Student Self-Service Mobile Access	Progressive Web App (PWA) with offline caching, mobile borrow reservations, digital student pass, 2D QR receipts	mobile-app/ cdm_library_mobile.jsx
Secure Account Password Recovery	Automated 6-digit Email OTP generation, 10-minute expiration check in SQLite, bcrypt password re-hashing	server/server.js otp_codes table
Turnstile Identity Verification	Dynamic 2D QR Receipt canvas generation (qrcode library) containing verified JWT transaction payload	QRReceiptModal qrcode package

2. Complete Technology Stack & Architecture

- **Frontend Desktop App:** Built using React 19, TypeScript, and Vite 6. Styled with Tailwind CSS v4 featuring Linear / Shadcn UI design tokens, Lucide React icons, and Motion micro-animations.
- **Mobile PWA Portal:** Built as a responsive Progressive Web Application utilizing Web App Manifest (manifest.webmanifest), Service Worker caching, and touch-optimized UI components.

- **Backend REST API:** Powered by Node.js (v25+) and Express.js (v5). Features custom JWT authentication middleware, CORS origin validation, and bcrypt password hashing.
- **Database Storage:** SQLite 3 using the high-performance synchronous driver better-sqlite3. Automatically creates relational schemas, indexes, and seeds records on startup.

Layer	Technologies & Libraries Used	Core Technical Functionality
UI & View Layer	React 19, TypeScript, Tailwind CSS v4, Lucide React	Single-Page Application (SPA), MARC 21 filtering, responsive layouts
State & Logic	React Hooks (useState, useMemo, useRef)	Memoized search algorithms, modal dialog management, form controls
Server & API	Node.js, Express.js v5, CORS	RESTful JSON API endpoints, error handling, route protection
Security & Auth	JSON Web Tokens (JWT), bcrypt (10 rounds)	Stateless session management, password hashing, 6-digit OTP verification
Database	SQLite 3, better-sqlite3	Relational tables, foreign keys, synchronous queries, automated seeds
Mobile PWA	Service Workers, Web Manifest, LocalStorage	Student portal, mobile borrow reservations, offline capability

3. Core Codebase Files & Implementation Details

A. Backend Engine: server/server.js

Contains Express server setup, REST routes, JWT token verification middleware, and password recovery endpoint logic.

```
// Example JWT Authorization Middleware in server/server.js function authenticateToken(req, res, next) { const authHeader = req.headers['authorization']; const token = authHeader && authHeader.split(' ')[1]; if (!token) return res.status(401).json({ error: 'Access denied. No token provided.' }); jwt.verify(token, JWT_SECRET, (err, user) => { if (err) return res.status(403).json({ error: 'Invalid or expired token.' }); req.user = user; next(); }); }
```

B. Database Schema & Storage: server/db.js

Uses better-sqlite3 to define relational schemas for books, students, librarians, transactions, reservations, and otp_codes.

```
-- Core Database Schemas in server/db.js CREATE TABLE IF NOT EXISTS books ( id TEXT PRIMARY KEY, title TEXT NOT NULL, author TEXT NOT NULL, isbn TEXT, category TEXT, cover TEXT, available INTEGER, total INTEGER, borrowCount INTEGER DEFAULT 0, publishYear INTEGER, callNo TEXT, institute TEXT, yearLevel TEXT, semester TEXT, marcTags TEXT ); CREATE TABLE IF NOT EXISTS otp_codes ( id INTEGER PRIMARY KEY AUTOINCREMENT, email TEXT NOT NULL, code TEXT NOT NULL, expires_at INTEGER NOT NULL, used INTEGER DEFAULT 0 );
```

C. Desktop Application SPA: src/app/App.tsx

Implements the main Librarian Desktop interface with real-time MARC 21 filtering using React useMemo, 3:4 aspect ratio covers with stateful fallbacks, and dense table controls.

```
// High-Performance Filter Algorithm in src/app/App.tsx const filtered = useMemo(() => { return books.filter(b => { const bookInst = (b.institute === "GENED" ? "ITE" : b.institute) || "ITE";
```

```
const matchInst = selectedInstitute === "All" || bookInst === selectedInstitute; const matchYear = selectedYear === "All Years" || b.yearLevel === selectedYear; const q = search.toLowerCase().trim(); const matchSearch = !q || b.title.toLowerCase().includes(q) || b.author.toLowerCase().includes(q) || b.isbn.includes(q); return matchInst && matchYear && matchSearch; }); }, [books, selectedInstitute, selectedYear, search]);
```

4. Defense Panel Q&A; Master Playbook

Use these exact technical responses during your capstone defense when questioned by panel members:

Q1: How does your system ensure academic cataloging standards across Colegio de Montalban?

Answer: We integrated official MARC 21 Tagging Standards (Machine-Readable Cataloging) alongside Library of Congress (LC) Call Numbers. Books are tagged with fields like Tag 001 (Control Number), Tag 020 (ISBN), Tag 082 (Dewey Classification), Tag 100 (Author), Tag 245 (Title), and classified under specific CDM institutes (ICS, ITE, IBE).

Q2: Why did you use SQLite and `better-sqlite3` instead of MySQL or PostgreSQL?

Answer: SQLite provides a lightweight, embedded, self-contained database engine. `better-sqlite3` operates synchronously in Node.js, removing async promise overhead for rapid local transactions, barcode scanning, and instant turnstile receipt verification while keeping infrastructure maintenance minimal.

Q3: How do you handle password security and account recovery?

Answer: Passwords are never stored in plain text; they are salted and hashed using `bcrypt` with 10 hashing rounds. For recovery, our system generates a 6-digit numeric OTP with a 10-minute expiration timestamp stored in SQLite (`otp\_codes`). Once verified, the user resets their password, which is immediately re-hashed with `bcrypt`.

Q4: How does the system prevent unauthorized access to API endpoints?

Answer: We implement stateless JWT (JSON Web Token) authentication middleware (`authenticateToken`). Upon login, the backend issues a signed JWT containing user credentials. Every protected HTTP request requires this token in the `Authorization: Bearer` header, which the backend verifies before granting access.

Q5: How do you handle UI performance when filtering hundreds of cataloged volumes?

Answer: We utilize React's `useMemo` hook to memoize filtering and sorting operations. The multi-criteria filtering algorithm only executes when filter state variables (`selectedInstitute`, `selectedYear`, `search`) change, preventing unnecessary DOM re-renders and maintaining 60 FPS UI responsiveness.

Q6: How does the Mobile PWA feature benefit Colegio de Montalban students?

Answer: The PWA allows students to search prescribed references, view live book availability, place mobile borrow reservations for desk pickup, and generate digital student passes on their smartphones without requiring app store installation.

Q7: How are lost book replacements handled in the system?

Answer: Librarians can update book status or log replacements directly via the system. When a replacement is processed, the system updates inventory counts (`total` and `available`), logs the fine or status change in the transaction ledger, and updates student status accordingly.

Q8: What happens if a book cover image fails to load or has a broken URL?

Answer: Our `BookCard` component features stateful image error handling (`imageError`). If an image fails to load, it automatically transitions to a custom typographic spine cover fallback styled with the book's institute code, title, author, and call number, preventing broken UI elements.

5. Team Defense Meeting Agenda & Preparation Checklist

Time / Phase	Agenda Item & Focus Area	Assigned Lead & Action Items
00:00 - 00:10	System Architecture & Database Overview	Review server/db.js schemas, tables, and relationships
00:10 - 00:25	Backend REST API & JWT Security Walkthrough	Review server/server.js endpoints, JWT middleware, OTP logic
00:25 - 00:40	Desktop SPA & MARC 21 Cataloging Demonstration	Demonstrate src/app/App.tsx filtering, cataloging, QR receipt generation
00:40 - 00:55	Mobile PWA Student Portal Walkthrough	Demonstrate mobile borrow reservations and student pass generation
00:55 - 01:10	Mock Panel Q&A; Drill	Practice Section 4 Panel Q&A; questions with all team members